

AutoTriager Shop

Human-AI co-design of an evidence-grounded incident workbench

Dahe Chen | CS 5588 Data Science Capstone, Challenge 2 | October 1, 2026

Abstract

A checkout failure can surface in several services even when one dependency initiated it. This project asks whether a novice on-call developer can use a read-only workbench to choose a defensible first service to inspect, verify the original telemetry, and defer judgment when the data are insufficient. We replaced an earlier benchmark replay with a running four-process shopping simulation, collected normal/fault/recovery records, implemented a bilingual evidence view and optional Gemini analysis, and compared methods on five newly captured local cases. A strong direct prompt and evidence-prioritized prompting both localized 3 of 4 injected faults and abstained on the clean control. The evidence therefore supports a working, inspectable prototype, not a localization gain or production readiness.

1. Problem and focused P1 task

User: a junior on-call developer handling a bounded checkout incident. The need is a design hypothesis; no user study has confirmed it. **Challenge 2 goal:** load one observed incident, apply a diagnostic method, show a candidate service with source-linked evidence or abstain, and allow a human to record a judgment. The semester product would support harder cross-service ambiguity and more realistic sources later. It never executes recovery actions.

Figure 1. Reconstruction of the human-selected investigation workflow before implementation.



2. Human Design and its limits

The human design fixed the decision boundary first: an engineer should inspect candidate services and evidence before deciding what to do. P1 combined case loading and existing-style analysis; evidence display made that path usable. Historical reviews, richer visualization, and optimization were lower priority; multiple unrelated systems were optional. The initial plan assumed that OpenRCA Bank cases established a suitable application scenario. They did not show a live shopping workflow, so their earlier scores are excluded from this report.

The revised acceptance gate requires separate running services, controlled normal/fault/recovery phases, labels isolated from the model, source-resolvable claims, a human veto, and comparison with a capable direct prompt. This directly addresses the instructor's questions: who uses the tool, where the need occurs, and why a prompt alone may suffice.

Earlier project discussions involved AI assistance. The Human Design here records the student's selected boundaries; it is not presented as an independently time-stamped, AI-free document.

3. AI Design: useful structure and a necessary correction

Earlier AI design advice proposed replaying OpenRCA Bank incidents through EviRCA-style evidence extraction, model reasoning, and a result view. That architecture did not itself establish a runnable shopping workflow. After the revised requirement, the coding assistant proposed a public case schema, UTC windows, stable IDs, trace parent links, deterministic first pass, budgeted evidence selection, optional LLM reasoning, citation validation, and a local review interface. It also found that official Astronomy Shop source could be pinned but not run here without WSL and Docker.

The AI proposal needed critical review. It had previously treated Bank benchmark results as evidence for this product, suggested a weak direct prompt as the comparator, and risked conflating a small local simulator with the official Shop. I rejected those shortcuts. A strong same-budget prompt is the meaningful comparator; local and official telemetry have separate provenance; a screenshot is interface evidence, not proof of diagnostic accuracy.

Figure 2. Reconstructed earlier AI proposal, not an executed shopping architecture: Bank replay supplied no live checkout or matched intervention.



4. Explicit Human-AI co-design

Decision	Human Design	AI proposal	Final co-design
Scenario	Read-only aid for a new on-call engineer	Replay benchmark incidents	Run a controlled shopping incident; exclude Bank scores
Data	Metrics, traces, logs where available	Normalize as compact text	Preserve numbers, UTC, service, trace/parent IDs, raw line refs
Retrieval	Find relevant observations	Evidence-first context	Rule-based top-48 selection; no claim of trained RAG
Grounding	Engineer checks before action	Cite IDs	Validate IDs, open raw lines, abstain when thin
Evaluation	Check injected labels	Weak prompt baseline	Add strong prompt, clean controls, explicit failure

The AI-only proposal specified a data path, but did not establish user value or fair evaluation. Human review required a live scenario, isolated labels, a strong direct-model comparator, and separate claims for the local simulator and official Shop.

These changes are design improvements because they make the experiment, user action, and evidence boundary inspectable. They are **not** measured improvements in engineer speed or model accuracy. The strong direct prompt tied the grounded prompt in valid-set fault localization, so a claim that the new architecture beats prompting would be unsupported.

5. Human and AI responsibilities

- The human selected the user decision, target setting, first P1 scope, comparison standard, and what counts as unacceptable evidence leakage.
- AI assisted source inspection, schema/adaptor code, test construction, UI implementation, and identification of failure modes.
- The human reviewer must decide whether a cited observation really supports a conclusion and whether any operational action is appropriate.

6. Data and multimodal processing

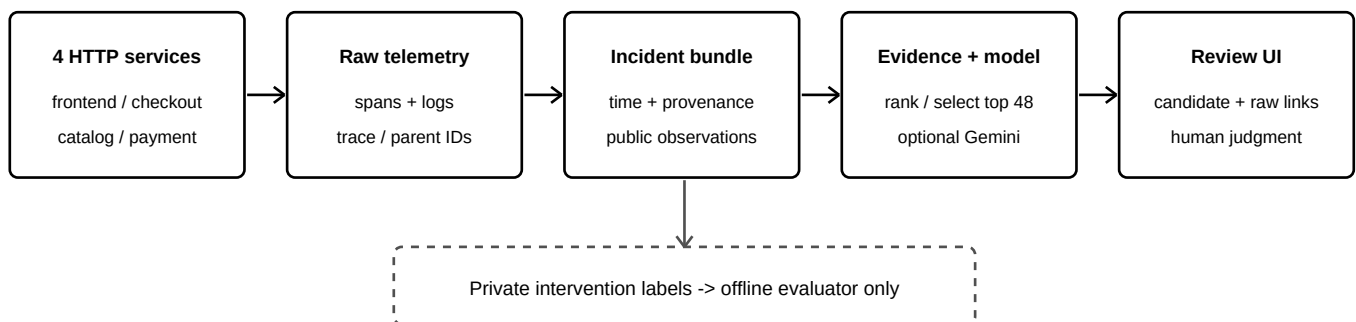
The evaluated source is this project's **native local shopping simulation**: four separate Python HTTP processes named frontend, checkout, catalog, and payment. A test order actually crosses process boundaries. Fault modes include payment rejection, payment capacity pressure, payment delay, catalog error, checkout error, and no fault. Each run records normal, injected, and recovery traffic. One capacity run had 24/24 normal HTTP 200 responses, 21/24 checkout errors during injection, and 24/24 recovery HTTP 200 responses.

The five newly captured bundles contain 260 span, 234 log, and 40 derived metric observations (534 total). Raw spans/logs are append-only NDJSON with service, UTC timestamp, trace ID, span ID, and parent-span ID. Error-rate and latency metrics are computed from normal versus alert spans; they are not independent Prometheus measurements. A public observation has a stable ID and a reference such as `raw/payment . spans . ndjson#L8`. Numeric values and topology are retained as structured fields rather than flattened into prose.

The intervention, expected service, traffic counts, and phase bounds reside in a separate `ground_truth.json` for offline scoring. Diagnostic code and UI load only `incident.json` and `observations.json`; the loader rejects answer fields in public input. Source-kind metadata distinguishes a captured local run from an official-Shop run or a synthetic unit-test fixture.

Figure 3. Final implemented architecture and isolated evaluator. The official-Shop adapter is staged, not live-verified.

Future adapter: official Astronomy Shop / Prometheus / Jaeger (not yet live-verified)



7. Methods and grounding

Python computes windows, HTTP counts, features, trace relationships, and raw-link checks. A deterministic baseline scores explicit errors/anomalies, caps duplicate records per signal kind, and uses bounded trace direction as a priority hint. Scores are not probabilities. It abstains when evidence is absent, only one kind supports the leader, or leaders tie. Gemini 3.5 Flash-Lite is optional: it sees at most 48 compact, public observations, returns a candidate/reason/evidence IDs/missing evidence, and is called only after local-user opt-in.

The grounded path prioritizes anomalous and error observations before prompting. This is metadata/rule-based retrieval, **not** a vector database, a trained RAG index, fine-tuning, or a multi-agent diagnosis. The validator rejects unsupported claims with unknown cited IDs and requires two cited signal kinds for a supported grounded output. The interface resolves citations to the original record. Referential integrity does not prove a citation is causally relevant.

8. Working application feature

The local Streamlit app selects a captured case, shows its provenance and symptom, runs the deterministic analysis, optionally calls Gemini, lists candidate services, exposes source records, and accepts a reasoned human review. English and Chinese UI text are available. The review form now requires an explicit judgment; it does not preselect 'accept.' The model path was exercised in the UI on a local capacity case: Gemini returned payment from 48 visible records in 1,257 ms. This is one interaction, not a held-out accuracy estimate.

● Gemini grounded analysis

Diagnosis

Evidence status: Supported hypothesis

Analysis method: gemini-grounded-v1

Model: gemini-3.5-flash-lite · API latency: 1257 ms · Observations sent to model: 48

Candidate services

First service to inspect: payment. An investigation priority, not a proven root cause.

1. payment

Reasoning: The payment service logs indicate capacity exhaustion and return HTTP 503 errors, causing dependent failures in checkout and frontend.

Cited evidence: log-payment-1, log-payment-2, log-payment-3, log-payment-4 · 10 more IDs; inspect the evidence table below

Figure 4. Genuine local application screenshot. The case came from this project's native simulator, not an official Astronomy Shop deployment. Model output is an investigation priority.

The separate storefront makes the scenario tangible. In the live smoke test, a normal test order returned HTTP 200; after the payment fault switch, the same endpoint returned HTTP 502; the fault was then reset. The frontend, checkout, catalog, and payment processes each wrote raw trace/log records. The screenshot below shows the fault response; the injected label was never passed to the analysis input.

Moon Mug

Local four-service shopping simulation.

\$19.99

Place test order

```
{
  "error": "checkout failed",
  "detail": {
    "error": "payment dependency failed",
    "detail": {
      "error": "payment rejected by service"
    }
  }
}
```

Figure 5. Actual local storefront fault smoke test. It demonstrates a running request path, not enterprise-scale realism.

9. Corrected five-case evaluation

Earlier development cases had a confounded delay intervention: payment slowed while the checkout timeout also changed. We excluded their scores. Before the five cases below were captured, the analysis rule was frozen and checkout's payment timeout fixed at 250 ms in all phases; the delay mode changes only payment response time. These cases were then run once through each Gemini configuration without tuning after the results. This is still a small local simulator, not an independent production or official-Shop estimate.

The injected service is held in a private evaluator label. Both methods used Gemini 3.5 Flash-Lite and a maximum 48-record context budget, but the prompt, selected evidence, and grounded validator differ. Client latency is one model call, not human resolution time. An API error is not scored as an abstention.

Gemini configuration	Fault localized	Clean abstain	Clean false attribution	Median API latency
Strong direct	3/4	1/1	0/1	1,509 ms
Evidence-prioritized	3/4	1/1	0/1	3,077 ms

Table 1. Measured five-case local-simulation comparison. The two methods tied on fault localization and clean abstention; no accuracy or speed gain is established.

Case-level audit

Case	Injected service	Strong direct	Grounded	Grounded result
Payment capacity	payment	payment	payment	correct
Catalog error	catalog	catalog	catalog	correct
Checkout error	checkout	checkout	checkout	correct
Payment delay	payment	checkout	abstain	miss: abstained
Clean control	none	abstain	abstain	correct

Table 2. Case-level outputs from four faults and one clean control; each mode ran once per case.

Failure: the checkout caller timed out around 260 ms, while payment completed an approximately 550 ms span with OK status. Strong direct incorrectly named checkout. The grounded path abstained rather than making that false attribution, but also missed the known payment intervention. This exposes a temporal-evidence gap, not superior localization.

All ten saved model outputs had nonempty, known citation IDs with resolvable raw links, including the clean case. A valid pointer does not establish causal relevance. The observed latency difference is based on five calls per configuration and is not a speed claim. Differences in selection, prompt, and validation prevent causal attribution of any behavior to retrieval alone.

10. Reproducibility and current boundary

The repository includes the four-process simulator, Streamlit app, schema, deterministic analyzer, Gemini client, capture adapter, tests, result CSV/summary, genuine local screenshots, and a public replay case without private fault labels. On Windows with Python 3.13, run `py -3.13 -m pip install -r requirements.txt`, `py -3.13 -m scripts.seed_example_case`, and `py -3.13 -m streamlit run app.py --server.address 127.0.0.1`. To generate a new run, execute `py -3.13 -m native_shop.run_experiment --case-id new-01 --mode payment_error`. Gemini requires a separately configured API key and may have quotas. Never commit the key.

The official OpenTelemetry Astronomy Shop source is pinned separately to release 3.1.0 at commit `dedc0178918e260823323b8d95005a8cb924b007`. A Prometheus/Jaeger capture adapter and protocol exist, and mocked adapter tests pass; **no official-Shop run or capture has occurred on this host**. Windows WSL features were enabled, but a restart plus working Docker runtime remain necessary. No upstream screenshot is presented as local execution.

Limits: the native topology is small; injected error logs can be more explicit than real incidents; five cases on one simulator are not external validation; no independent on-call user study or time-to-resolution measure exists; source-link validity is weaker than causal validity; and Gemini output can change with model/API versions. A repository URL remains pending verified publication. The September 30 course deadline has passed; this report does not claim on-time submission or an instructor grade.

11. Next challenge and three uses of the same artifact

Challenge 3: capture a controlled incident on a verified official Shop deployment and show a trace-linked comparison between an initiating service and downstream symptoms. **Challenge 4:** test ambiguity under missing/delayed signals; consider concurrent faults only after a valid labeling protocol. **Challenge 5:** integrate reproducible setup, feedback, and a live demonstration. The course gets one complete feature and a candid evaluation; research gets a falsifiable comparison and future holdout/ablation plan; a hackathon gets a real fault-to-evidence interaction. None of these implies a publishable accuracy improvement yet.

Primary resources

[1] Course-provided CS 5588 Challenge 2 report requirements, September 2026.

[2] OpenTelemetry Demo source and Apache-2.0 license:

<https://github.com/open-telemetry/opentelemetry-demo/tree/dedc0178918e260823323b8d95005a8cb924b007>

[3] OpenTelemetry Docker deployment: <https://opentelemetry.io/docs/demo/docker-deployment/>

[4] Prometheus range-query API: <https://prometheus.io/docs/prometheus/latest/querying/api/>

[5] Gemini API model and pricing documentation: <https://ai.google.dev/gemini-api/docs/pricing>

[6] Context only, not test data: OpenRCA <https://microsoft.github.io/OpenRCA/>; EviRCA <https://arxiv.org/abs/2609.19825>